

Practical 4 : Adapter Pattern

Part B: Analysis Questions

Q1. Why can't the notification system directly use the legacy email service?

The legacy email service has a different interface from what the notification system expects. Directly using it would tightly couple the system to the legacy implementation, making future changes difficult. The Adapter Pattern converts the legacy interface into the expected interface without modifying the existing code.

Q2. What problems arise when two classes have different interfaces?

Different interfaces prevent direct communication between classes. This results in compatibility issues, duplicated conversion logic, increased maintenance effort, and reduced flexibility when integrating new components.

Q3. What happens if multiple old systems need integration?

Without the Adapter Pattern, the client code would need separate logic for every legacy system, making it complex and difficult to maintain. Using adapters allows each legacy system to have its own adapter while the client interacts with a single common interface.

Q4. Is modifying third-party code a good practice? Why?

No. Modifying third-party code is not recommended because updates from the original provider may overwrite your changes, increase maintenance costs, and introduce unexpected bugs. Using an adapter preserves the original code while enabling compatibility.

Q5. Which software quality attributes are affected?

The Adapter Pattern improves several software quality attributes:

- Maintainability: Changes are isolated within the adapter.
- Reusability: Existing classes can be reused without modification.
- Flexibility: New implementations can be added easily.
- Scalability: Multiple legacy systems can be integrated through separate adapters.
- Extensibility: New adapters can be introduced without changing client code.

Part C: Problems in Existing Design

1. The new notification system expects a `sendNotification(String message)` method, but the legacy email service provides `sendMail(String text)`, so their interfaces are incompatible.
2. The notification system cannot directly use the legacy email service because the method names and expected interface are different.
3. Modifying the existing legacy email service to match the new interface is not recommended because it may be third-party or already used by other systems.

Practical 5 : Facade

Part B: Analysis Questions

Q1. Why use the Facade Pattern?

The Facade Pattern provides a simple interface to a complex subsystem, making the client code easier to use.

Q2. What problem does it solve?

It reduces coupling by hiding the complexity of multiple subsystem classes behind a single interface.

Q3. Can subsystem classes still be used directly?

Yes. The facade is optional and provides a simpler way to use the subsystem, but direct access is still possible if needed.

Q4. Where is the Facade Pattern used?

- Banking systems
- Student management systems
- E-commerce checkout
- Hotel booking systems
- Library management systems

Q5. Which software quality attributes improve?

- Maintainability
- Readability
- Reusability
- Simplicity
- Low coupling

Part C: Problems in Existing Design

1. Client interacts with multiple subsystem classes.
2. Registration process is scattered across different classes.
3. Changes to any subsystem require changes in the client code.

Practical 6 : Decorator Pattern

Part B: Analysis Questions

Q1. Why are many report classes being created?

Many report classes are created because each new feature (such as PDF, Watermark, or Signature) requires a separate subclass. As more features are added, the number of classes increases rapidly.

Q2. What happens when a new feature is added?

A new subclass must be created or existing classes must be modified. This increases code complexity and makes the system harder to maintain.

Q3. Why does inheritance become difficult to maintain?

Inheritance leads to a large number of subclasses for different feature combinations. Managing and updating these classes becomes difficult as the application grows.

Q4. How can the Decorator Pattern add features dynamically?

The Decorator Pattern wraps an existing object with decorator classes. Each decorator adds a new feature at runtime without changing the original class.

Q5. Which software quality attributes are affected?

The Decorator Pattern improves:

- Maintainability
- Flexibility
- Reusability
- Extensibility
- Low Coupling

Part C: Problems in Existing Design

Identify at least three problems.

1. A separate subclass is required for every new report feature, resulting in too many classes.

2. Combining multiple features (such as PDF, Watermark, and Signature) requires creating additional subclasses, increasing code duplication.
3. The design is difficult to maintain because adding or changing features often requires modifying existing classes instead of extending functionality dynamically.

Practical 7 : Decorator Pattern

Part B: Analysis Questions

Q1. Why are many report classes being created?

Because each new feature requires creating another subclass, leading to many similar classes.

Q2. What happens when a new feature is added?

A new subclass must be created or existing subclasses modified, increasing maintenance effort.

Q3. Why does inheritance become difficult to maintain?

As features increase, the number of subclasses grows rapidly, making the code difficult to understand and maintain.

Q4. How can Decorator add features dynamically?

Decorator wraps an existing object and adds new functionality at runtime without changing the original class.

Q5. Which software quality attributes are affected?

- Maintainability
- Reusability
- Flexibility
- Extensibility
- Low coupling

Part C: Problems in Existing Design

1. Too many subclasses are required.
2. Difficult to combine multiple features.
3. Changes require creating or modifying many classes.